

```

model.add(Dense(4, input_dim=2, activation='relu'))

model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])

model.fit(X, y, epochs=500, verbose=0)

predictions = model.predict(X)

print("\nPredictions:")
for i in range(len(X)):
    print(f"Input: {X[i]} -> Predicted: {round(predictions[i][0])}")

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 0s 90ms/step

```

```

Predictions:
Input: [0 0] -> Predicted: 0
Input: [0 1] -> Predicted: 1
Input: [1 0] -> Predicted: 1
Input: [1 1] -> Predicted: 0

```

✓ Lab 6 | 26 Feb

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report
from sklearn.feature_extraction.text import TfidfVectorizer

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

df = pd.read_csv('/spam_or_not_spam.csv')
X_text = df['email'].fillna('')
y = df['label']

vectorizer = TfidfVectorizer(max_features=5000)
X = vectorizer.fit_transform(X_text).toarray()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# 4. Build Model
# =====
def create_model(lr):
    model = Sequential()
    model.add(Dense(64, activation='relu', input_shape=(X.shape[1],)))
    model.add(Dense(32, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model

```

```

# =====
# 5. Training + Graph
# =====
learning_rates = [0.01, 0.001, 0.005]

for lr in learning_rates:

    print("\n=====")
    print("Training with LR =", lr)

    model = create_model(lr)

    history = model.fit(
        X_train, y_train,
        epochs=200,
        batch_size=32,
        validation_split=0.1,
        verbose=0
    )

    plt.figure(figsize=(8,5))
    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.title(f"Training vs Validation (LR={lr})")
    plt.xlabel("Epochs")
    plt.ylabel("Value")
    plt.legend()
    plt.grid(True)
    plt.show()

    y_pred = (model.predict(X_test) > 0.5).astype("int32")

    # =====
    # Metrics
    # =====
    print("Accuracy :", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(y_test, y_pred))
    print("Recall    :", recall_score(y_test, y_pred))
    print("F1 Score :", f1_score(y_test, y_pred))
    print("\nClassification Report:\n", classification_report(y_test, y_pred))

    # =====
    # Confusion Matrix Heatmap
    # =====
    cm = confusion_matrix(y_test, y_pred);

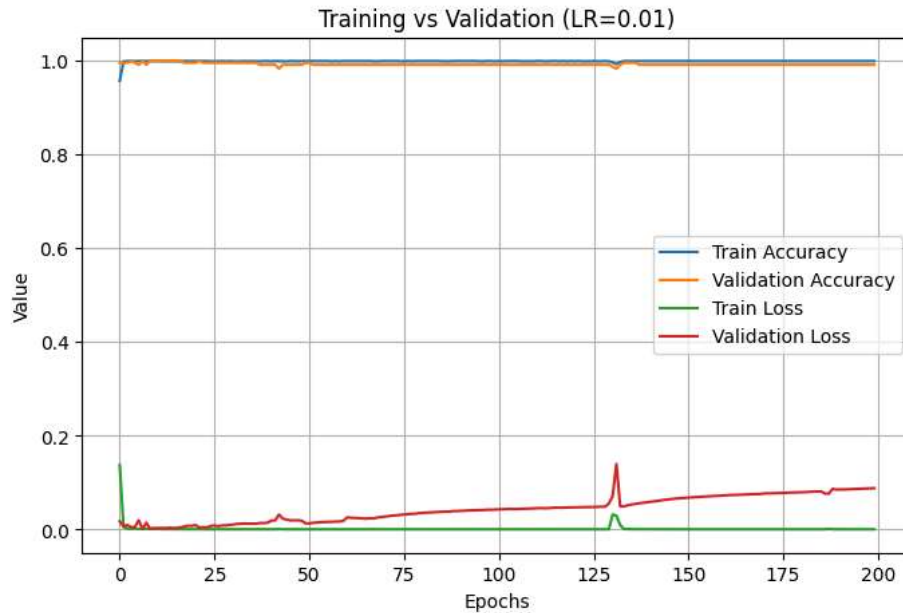
    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt='d',
                xticklabels=['Ham', 'Spam'],
                yticklabels=['Ham', 'Spam'])
    plt.title(f"Confusion Matrix (LR={lr})")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()

```


=====

Training with LR = 0.01

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input\_shape`/`input\_dim`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)



19/19 ————— 0s 5ms/step

Accuracy : 0.9783333333333334

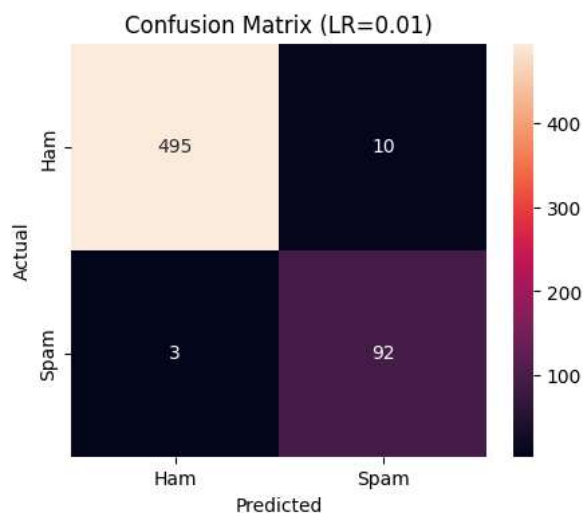
Precision: 0.9019607843137255

Recall : 0.968421052631579

F1 Score : 0.934010152284264

Classification Report:

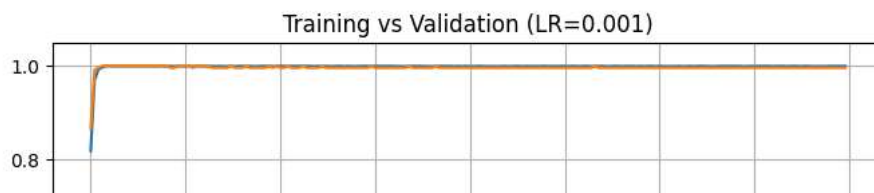
	precision	recall	f1-score	support
0	0.99	0.98	0.99	505
1	0.90	0.97	0.93	95
accuracy			0.98	600
macro avg	0.95	0.97	0.96	600
weighted avg	0.98	0.98	0.98	600

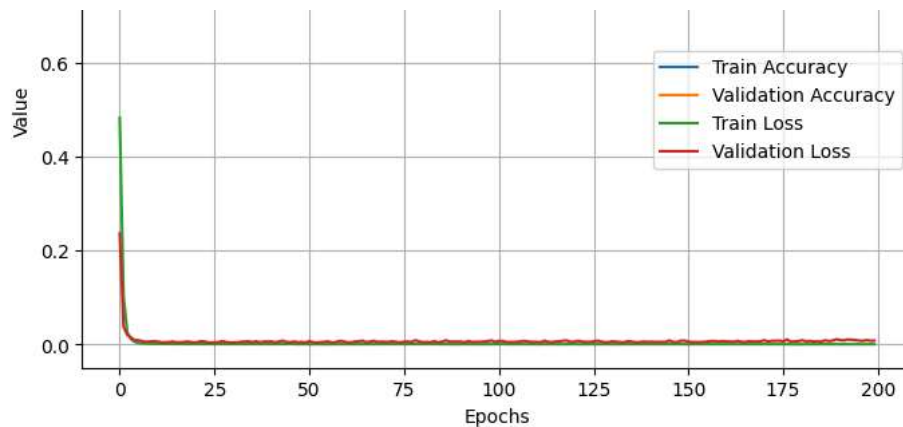


=====

Training with LR = 0.001

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input\_shape`/`input\_dim`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)





19/19 ————— 0s 5ms/step

Accuracy : 0.9883333333333333

Precision: 0.9680851063829787

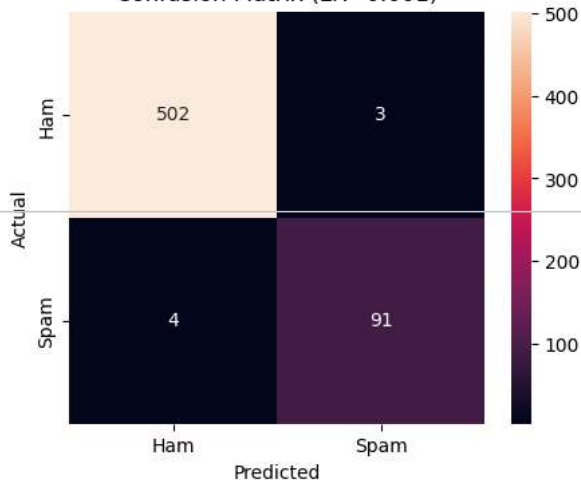
Recall : 0.9578947368421052

F1 Score : 0.9629629629629629

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	505
1	0.97	0.96	0.96	95
accuracy			0.99	600
macro avg	0.98	0.98	0.98	600
weighted avg	0.99	0.99	0.99	600

Confusion Matrix (LR=0.001)

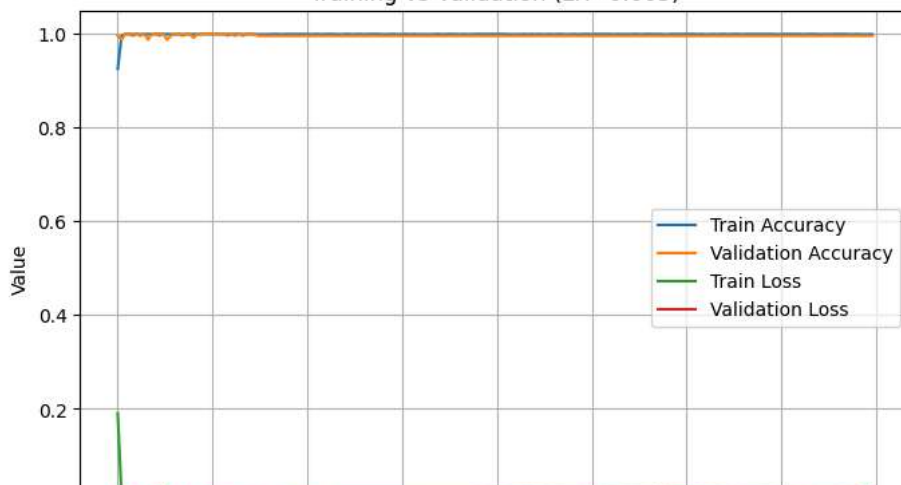


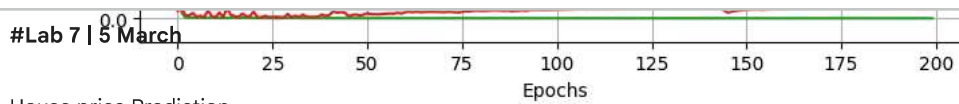
=====

Training with LR = 0.005

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input\_shape`/`input\_dim`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Training vs Validation (LR=0.005)





House price Prediction

19/19 0s 5ms/step

Accuracy : 0.9883333333333333

```
# =====
# 1. Import Libraries
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# =====
# 2. Load Dataset as DataFrame
# =====
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# Print Columns
print("\nColumns in Dataset:")
print(df.columns)

# =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# 5. Feature Scaling (VERY IMPORTANT)
# =====
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 6. Build Multilayer Model
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1) # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

# =====
# 7. Train Model
# =====
history = model.fit(
    X_train, y_train,
```

```

        validation_split=0.1,
        epochs=100,
        batch_size=32,
        verbose=1
    )

# =====
# 8. Evaluate Model
# =====
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# =====
# 9. Plot Training Curve
# =====
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# =====
# 10. Actual vs Predicted Plot
# =====
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()

```